Introduction to Machine Learning

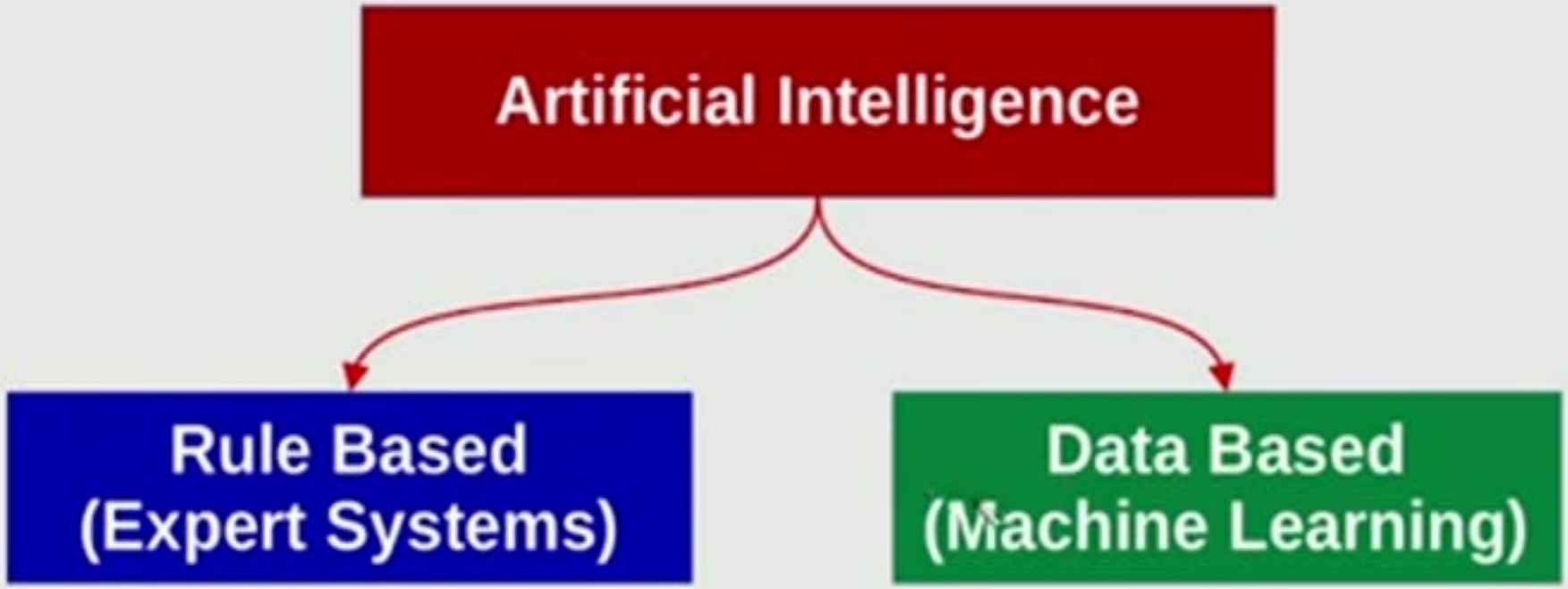
CSAI 253 - Spring 2024

Prepared By
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Instructors:

Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg
Dr. Mohamed Maher Ata
momaher@zewailcity.edu.eg




Artificial Intelligence
Rule Based
(Expert Systems)
Data Based
(Machine Learning)

What is Machine Learning?

Machine Learning:

- Designing algorithms that can learn patterns from data (and exploit them)
- **Approach**: human supplies training examples, the machine learns
- **Example:** Show the machine a bunch of spam and legitimate emails and let it learn to predict if a new email is spam or not

What is Machine Learning?

Machine Learning primarily uses the statistically motivated approach

- No hand-crafted rules - subtle pattern nuances are often be difficult to specify
- Instead, let the machine figure out the rules **on its own** by *building statistical models of the data*

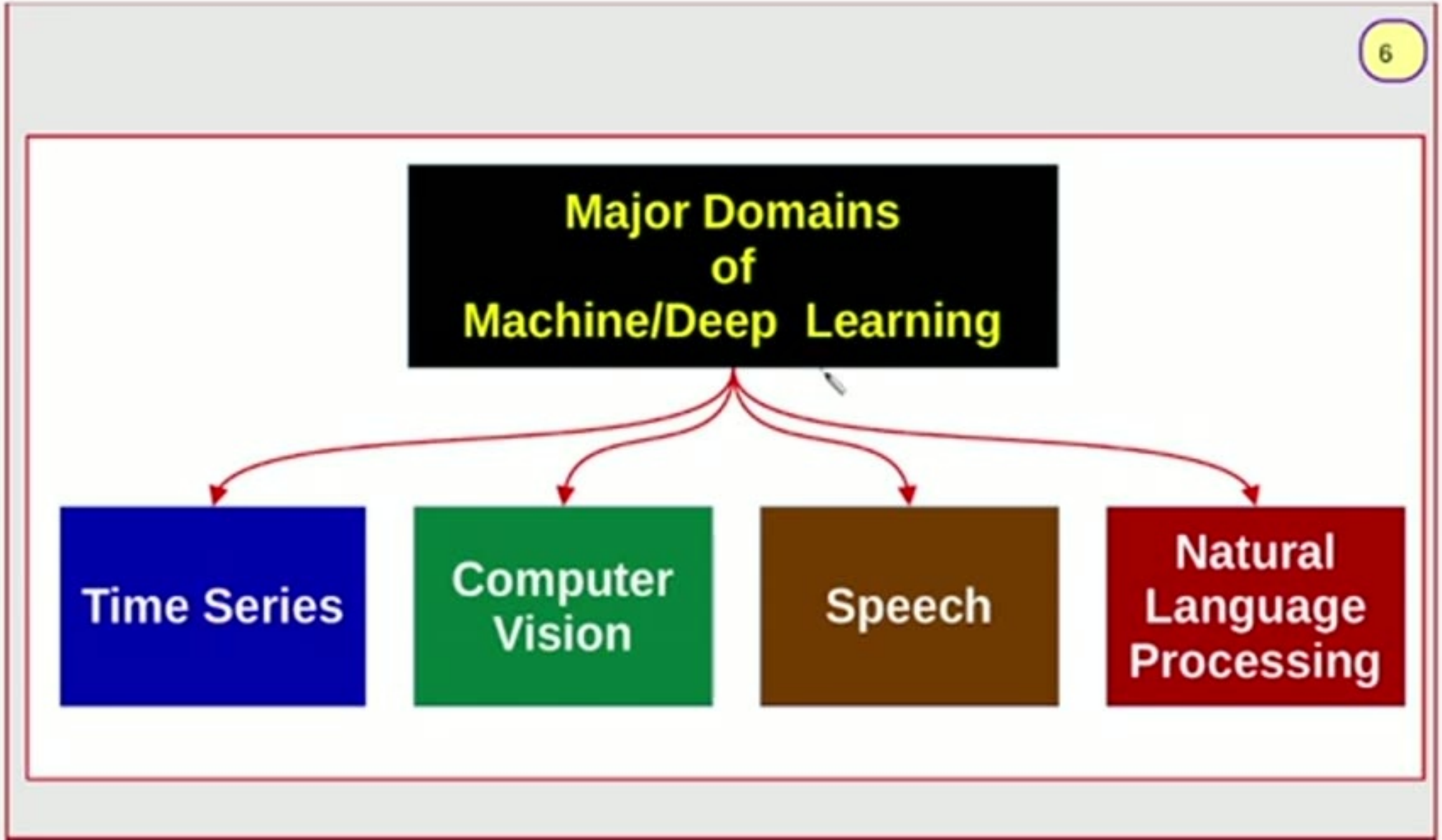
Desirable Property: Generalization

- The model shouldn't overfit on the training data
- **It should generalize** well on unseen (future) test data

Machine Learning in the real-world

Broadly applicable in many domains (e.g., finance, robotics, bio-informatics, vision, natural language, etc.). Some applications:

- Spam filtering
- Machine Translation (e.g., Google Translate)
- Speech/handwriting recognition
- Object detection/recognition
- Weather prediction
- Classifying DNA sequences
- Stock market analysis
- Automatic vehicle navigation
- Search engines (e.g, Google)
- Credit-card fraud detection
- Webpage clustering (e.g., Google News)
- .. and many more


Major Domains
of
Machine/Deep Learning
Time Series
Computer
Vision
Speech
Natural
Language
Processing

Programming Languages, Platforms, and Libraries

Popular Languages for ML/DL

- Python
- R
- Matlab

Popular Platforms for ML/DL

- Tensor Flow
- Pytorch

Popular Libraries (for python)

- SciKit Learn
- SciPy
- NLTK
- Spacy
- Gensim
- OpenCV

Major Machine Learning Paradigms

Nomenclature: **x** denotes an input, **y** denotes output/label/prediction

[1] Supervised Learning: learning with a teacher

- Given: N labeled training examples $\{(x_1, y_1), \ldots, (x_N, y_N)\}$
- Goal: learn mapping **f** that predicts label y for a test example x
- Example: Spam classification, webpage categorization

[2] Unsupervised Learning: learning without a teacher

- Given: a set of N unlabeled inputs $\{x_1, \ldots, x_N\}$
- Goal: learn some intrinsic structure in the inputs (e.g., groups/clusters)
- Example: Automatically grouping news stories (Google News)

Major Machine Learning Paradigms

[3] Reinforcement Learning: learning by interacting

- Given: an agent acting in an environment (having a set of states)
- Goal: learn a policy (state to action mapping) that maximizes agent's reward
- Example:
- **Automatic vehicle navigation**, (computer) learning to **play Chess**

Supervised Learning

Given: N labeled training examples $\{(x_1, y_1), \ldots, (x_N, y_N)\}$

Goal: learn a model that predicts the label y for a test example x

Assumption: The training and the test examples are drawn from the same data distribution

Things to keep in mind:

- No single learning algorithm is universally good ("no free lunch")
- Different learning algorithms work with different assumptions
- Generalization is particularly important for supervised learning
-

sheet has just been saved...

Supervised Learning

f : x → y { f is a function from X to Y }

Classification:

y is a discrete variable

Discrete variable: takes a value from a discrete set $y \in \{1, \ldots, K\}$

Example: Category of a web page (sports, politics, business, science, etc.)

Regression:

y is a real-valued variable

Example: Price of a stock

Supervised Learning: Classification

Problem Types:

[1] Binary Classification:

- y is binary (two classes: 0/1 or -1/+1)
- Example: Spam Filtering (tell whether this email is spam or legitimate)

[2] Multi-class Classification:

- y is discrete with one of K > 2 possible values
- Example: Predicting your class grade (e.g., A, A−, B+, B, B−, other)
-

Supervised Learning: Classification

[3] Multi-label Classification:

When y is a vector of discrete variables

Each input x has multiple labels

Each element of y is one label (individual labels can be binary/multi-class)

Example: Image annotation (each image can have multiple labels)

[4] Structured Prediction: When y is a vector with a structure

Elements of y are not independent but related to each-other

Example: Predicting parts-of-speech (POS) tags for a sentence

Input	The	man	ate	the	really	tasty	sandwich
Output	DET	NOUN	VERB	DET	ADV	ADJ	NOUN

Supervised Learning: Regression

Problem Types:

Univariate Regression:

y is a single real-valued number

Example: Predicting the future price of a stock

Multivariate Regression:

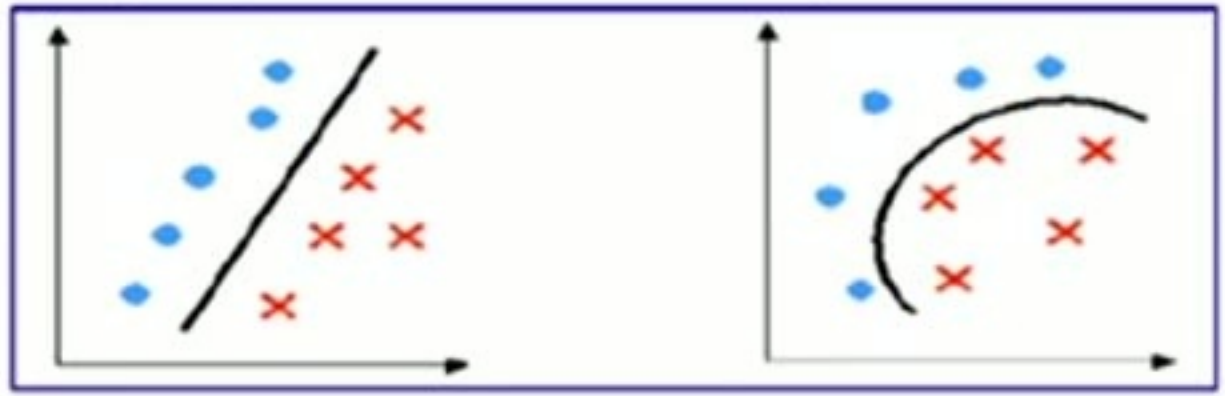
y is a real-valued vector

Each element of y tells the value of one response variable

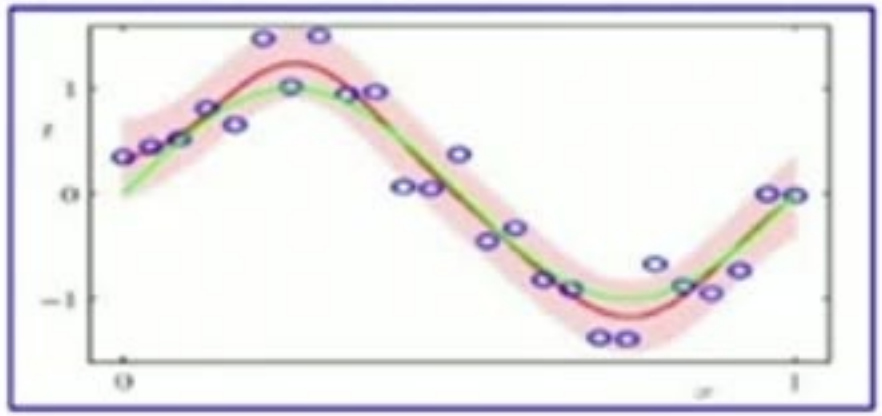
Example: Torque values in multiple joints of a robotic arm

Classification vs Regression

Classification is about finding separation boundaries (linear/non-linear):

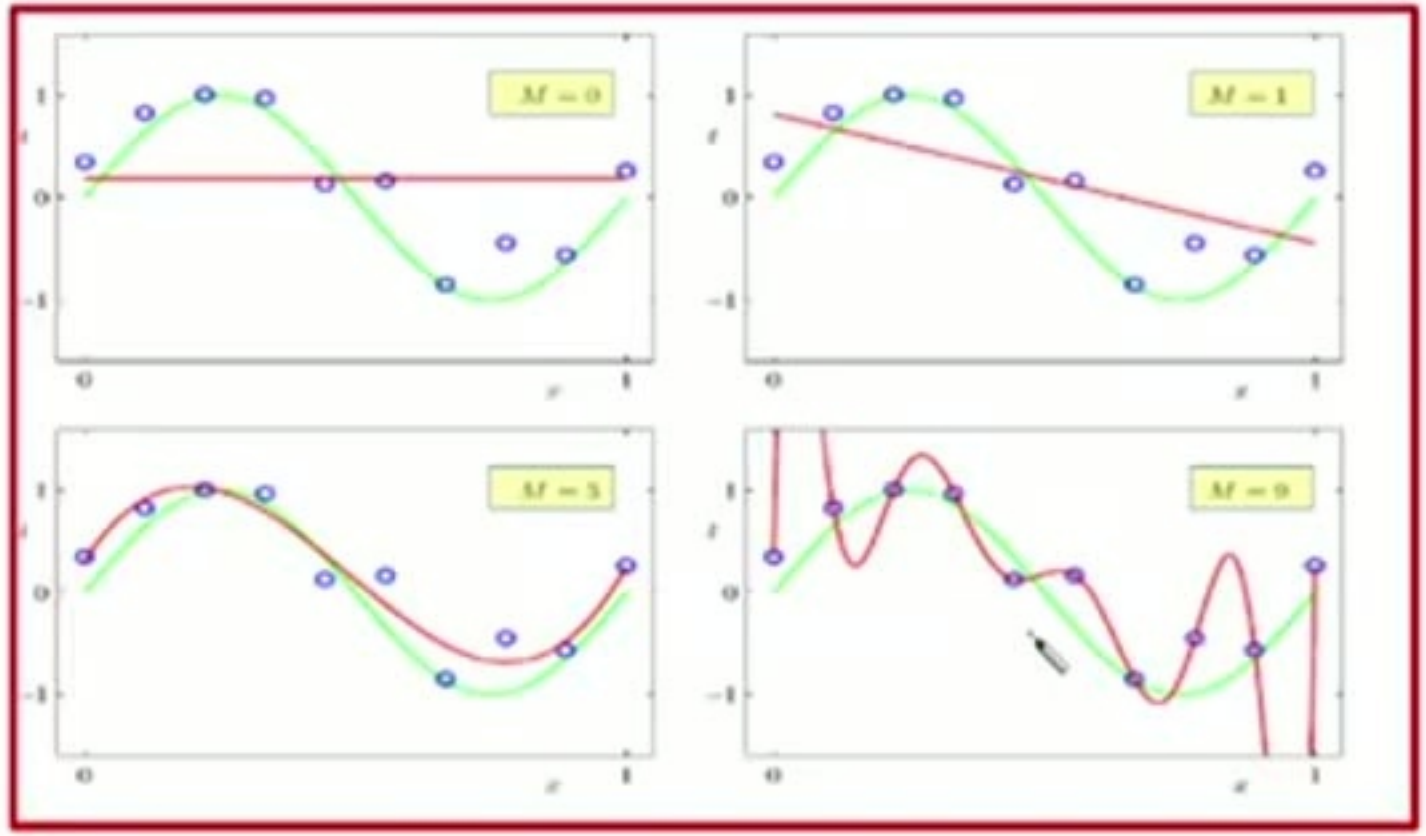


Regression is more like fitting a curve/surface to the data:



Data Overfit and Underfit

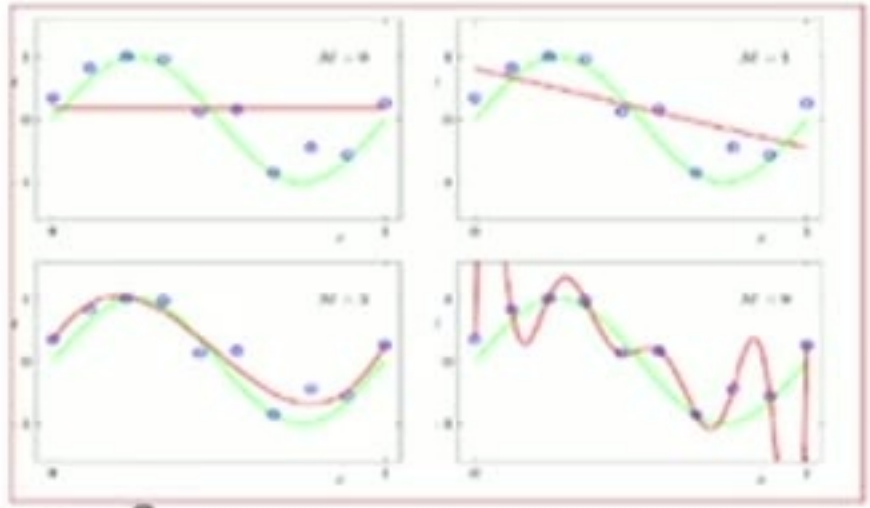
The X axis is the input. The Y axis is the response.



Data Overfit and Underfit

Questions:-

- Which of the four red curves fits the data (blue dots) best?
- Which curve is expected to generalize the best?

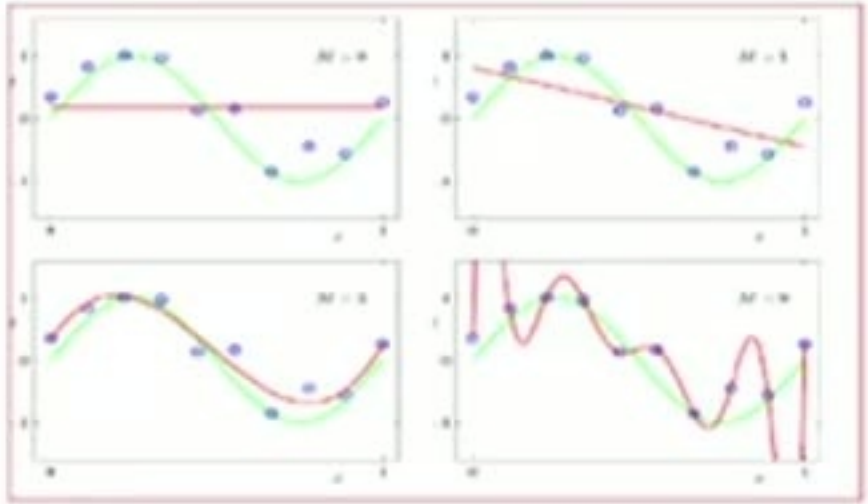


Are they both the same? If yes, why? If no, why not?

Data Overfit and Underfit

Questions:-

- Which of the four red curves fits the data (blue dots) best?
- Which curve is expected to generalize the best?



Are they both the same? If yes, why? If no, why not?

Lesson: Simple models should be **preferred** over complicated models

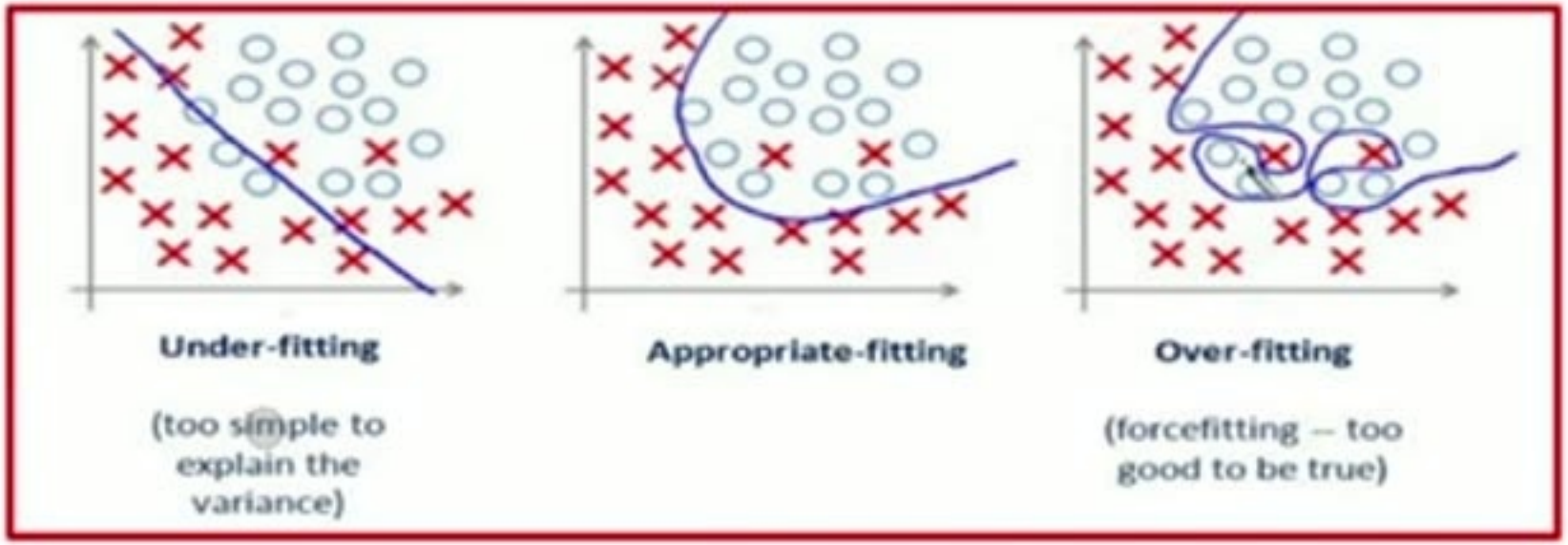
Simple models can prevent overfitting

Too simple a model can underfit (e.g., M = 0 above)

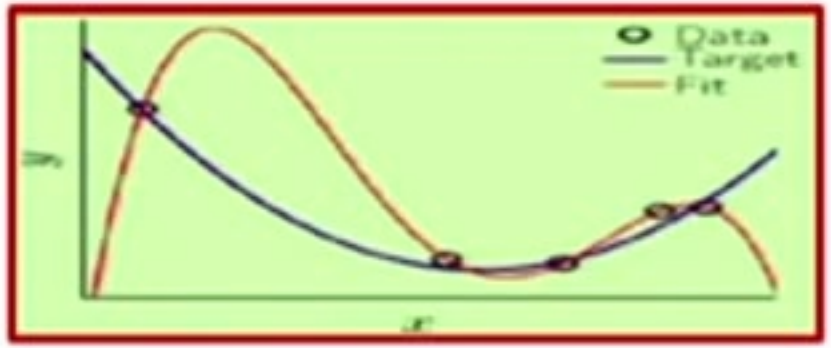
General guideline: Choose a model *not-too-simple, yet not-too-complex*

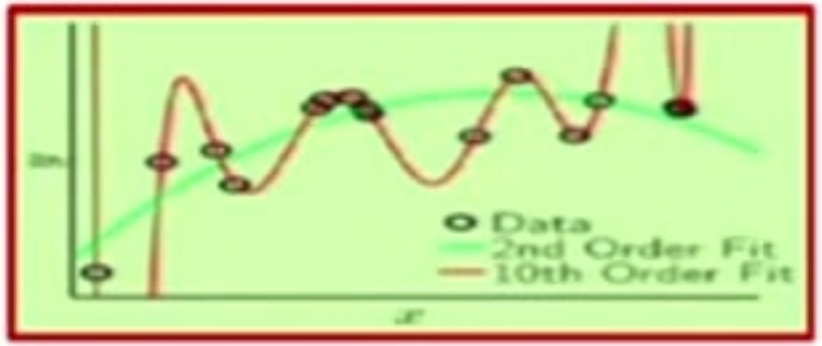
Data Overfit and Underfit

Classification



Regression





Reinforcement Learning (RL)

- Unlike supervised/unsupervised learning, RL does not receive examples
- Rather, it learns (gathers experience) by interacting with the world
- Defined by an agent and an environment the agent acts in
- **Agent** has a set **A** of actions, **environment** has a set **S** of states
- **Goal**: Find a **sequence of actions** by the agent that **maximizes its reward**
- **Output**: A policy which maps states to actions
- RL problems always include time as a variable
- **Example** problems: **Chess**, **Robot control**, **autonomous driving**

Other Paradigms: Semi-supervised Learning

- Supervised Learning requires labeled data (the more, the better!)
- **Problem 1:** Labeling is expensive (usually done by humans)
- **Problem 2:** Sometimes labels are really hard to get
- How can we learn well even with small amounts of labeled data?
- One answer: Semi-supervised Learning
- Using small amount of labeled + plenty of (freely available) unlabeled data

Other Paradigms: Active Learning

- Similar motivation as semi-supervised learning (saving data labeling cost)
- Standard supervised learning is passive
  - Learner has no choice for the data it has to learn from
  - Not all labeled examples are really informative
  - Spending labeling efforts on uninformative examples isn't really worth it
- Active Learning: allows the learner to ask for specific labeled examples
  - .. the ones it considers the most informative
- Active Learning can lead to **several benefits:**
  - Less labeled data needed to learn
  - Better classifiers

Data Sets



Annotated and unannotated data
Do you have labels for your Dataset ???

Data Representation

- Data has form: $\{(x_1, y_1), \ldots, (x_N, y_N)\}$ (labeled), or $\{x_1, \ldots, x_N\}$ (unlabeled)
- Each example X is a set of (numeric) **features** / attributes / dimensions
- X is commonly represented as a D × 1 vector
  - Representing a 28 × 28 image: x can be a 784 × 1 vector of pixel values
  - Representing a text document: x can be a vector of word-counts of words appearing in that document

Some Notations

- $\mathbb{R}^D$ denotes the set of all $D \times 1$ real-valued column vectors
- $\mathbf{x} \in \mathbb{R}^D$ denotes a $D \times 1$ real-valued column vector
- $\mathbf{x}^T$ denotes the transpose of $\mathbf{x}$, a $1 \times D$ *row vector*
- $\mathbb{R}^{N \times D}$ denotes the set of all $N \times D$ real-valued matrices
- $\mathbf{X} \in \mathbb{R}^{N \times D}$ denotes an $N \times D$ real-valued matrix
- Supervised Learning: Often, we write $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$ as $(\mathbf{X}, \mathbf{Y})$
  - $\mathbf{X}$ is an $N \times D$ matrix
  - Each row of $\mathbf{X}$ denotes an example, each column denotes a feature
  - x_{ij} denotes the j-th feature of the i-th example
  - $\mathbf{Y}$ is an $N \times 1$ vector. Row i denotes the label of the i-th example

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{pmatrix} = \begin{pmatrix} x_{11} \cdots x_{1D} \\ \vdots \quad \vdots \\ x_{N1} \cdots x_{ND} \end{pmatrix}$$

$$\mathbf{Y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix}$$

Non-Numeric Data Representation

- What if the data has a non-numeric representation (e.g. Tennis Playing example)
- Features are nominal (Low/High, Yes/No, Overcast/Rainy/Sunny, etc.)
- Features with only 2 possible values can be represented as 0/1
- *What about features having more than 2 possible values?*
- **Can't we just** ***map Sunny to 0, Overcast to 1, Rainy to 2***?

P	Play
N	No Play
Out	Outlook
T	Temperature
R	Humidity

Y	Out	T	R
P	Sunny	Low	Yes
N	Sunny	High	Yes
N	Sunny	High	No
P	Overcast	Low	Yes
P	Overcast	High	No
P	Overcast	Low	No
N	Rainy	Low	Yes
P	Rainy	Low	No

⇒

Y	Out	T	R
1	?	0	1
0	?	1	1
0	?	1	0
1	?	0	1
1	?	1	0
1	?	0	0
0	?	0	1
1	?	0	0

Data Representation

- Well, we could map Sunny to 0, Overcast to 1, Rainy to 2..
- But such a mapping may not always be appropriate
  - Imagine color being a feature in some data
  - Let's code 3 possible colors as **Red=0**, **Blue=1**, **Green=2**
  - This implies Red is more similar to Blue than to Green !
- **Solution**: For a feature with K > 2 possible values, we usually **create K binary features, one for each possible value**

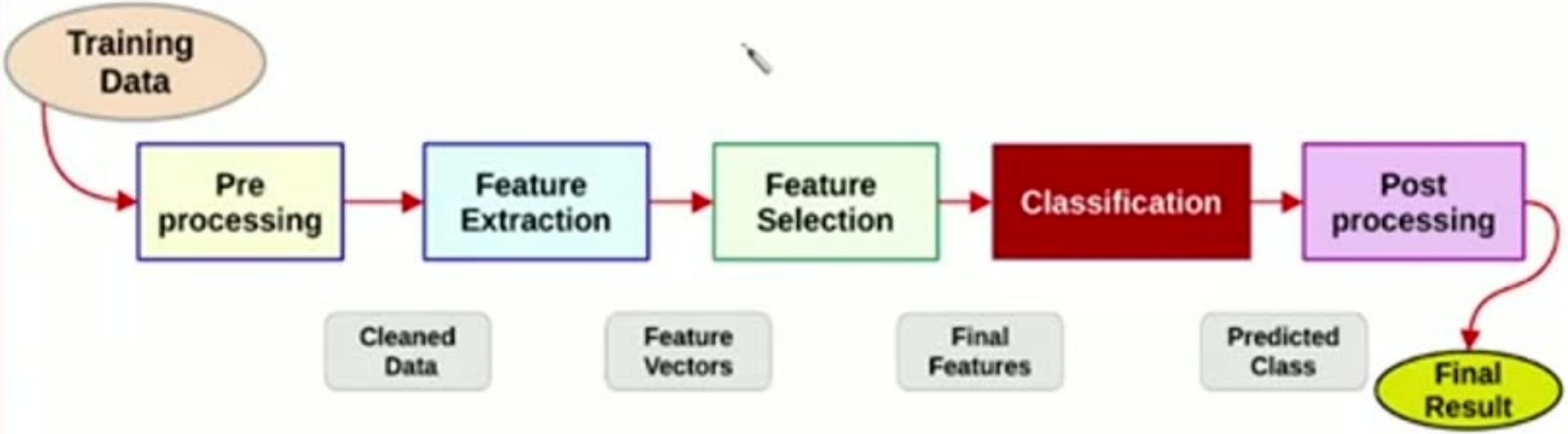
Replace
Out Outlook

With
S Sunny
O Overcast
R Rainy



The Complete Machine Learning Classification Process

The Complete ML Classification Process



Pre processing

Preprocessing is *Problem dependent and data dependent*

It may include:
Gap filling

- Thinning
- Contour
- Binarization
- Noise Removal
- Color to Gray
- Opening touched points
- Resizing
- Normalization
- ……

Feature Extraction

Features Types are is *Problem dependent*

Features may include:

- Center of Gravity location,
- Aspect Ratio,
- Length and Width,
- Number of intersections,
- Moments,
- Geometrical Features ,
- Colors,
- Transformations
- ……..

Feature Invariants

Features Invariants are *Problem dependent*

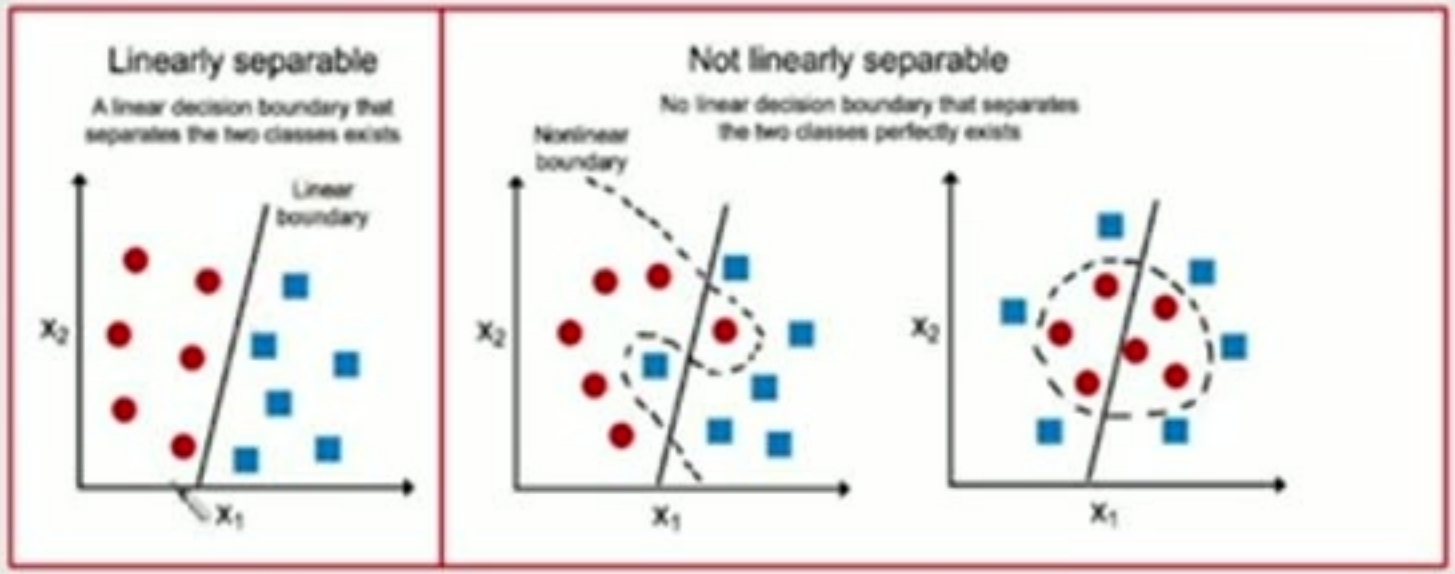
- Rotation Invariant
- Scaling Invariant
- Translation invariant
- Mirroring Invariant

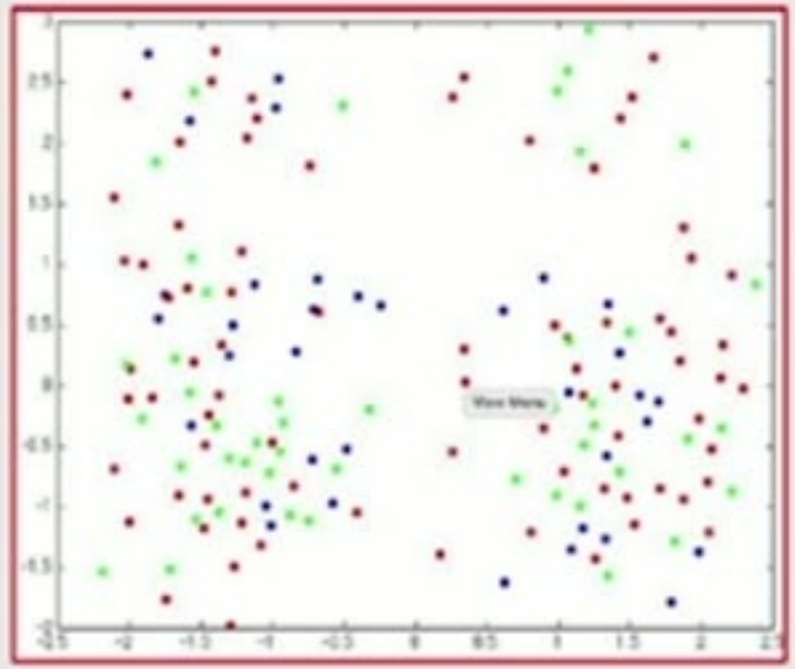
Classification

Classification Techniques are based on Data separability nature

From the separability point of view **data can be classified into** :

- **Linearly Separable** → **(Use Linear classifier)**
- **Non-Linearly Separable** → **(Use Non-Linear classifier)**
- **Non-Separable** → **(Use Kernels with Classifiers)**

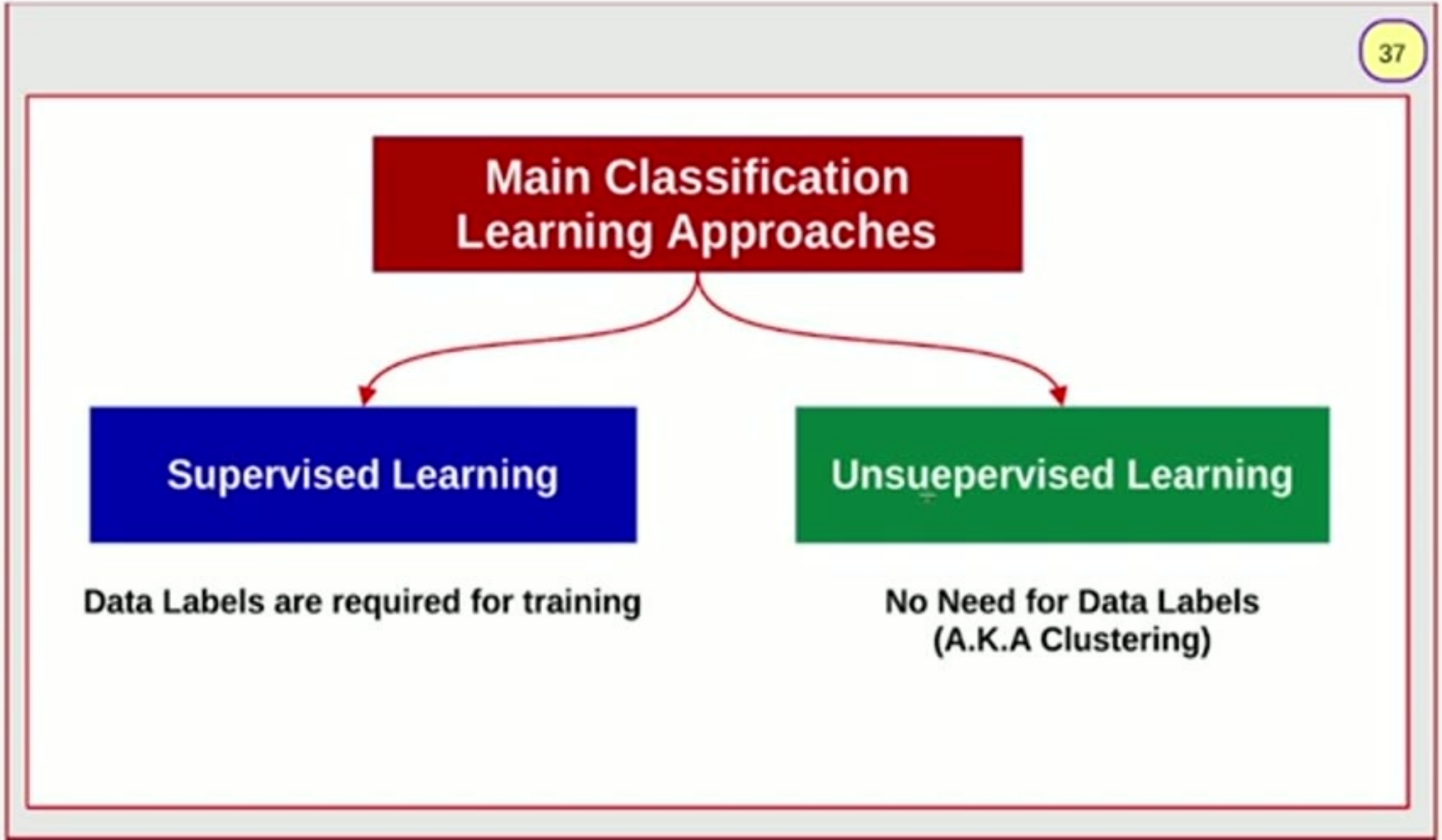

Linearly separable
A linear decision boundary that separates the two classes exists
Linear boundary
x_2
x_1
Not linearly separable
No linear decision boundary that separates the two classes perfectly exists
Nonlinear boundary
x_2
x_1
x_2
x_1



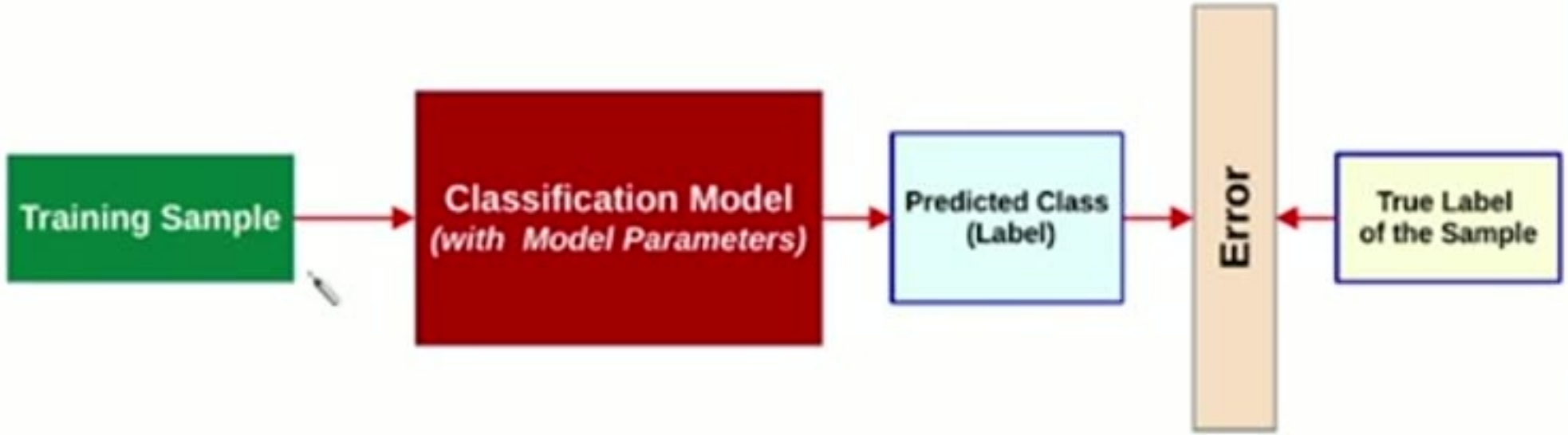
Classification

Examples of Commonly used classifiers:

- Linear Regression
- Logistic Regression
- Neural Network
- Support Vector Machine
- Hidden Markov Model
- Decision Trees
- Bayesian classifier
- .....


Main Classification Learning Approaches
Supervised Learning
Unsuepervised Learning
Data Labels are required for training
No Need for Data Labels (A.K.A Clustering)

How Training Takes Place to learn the model ?

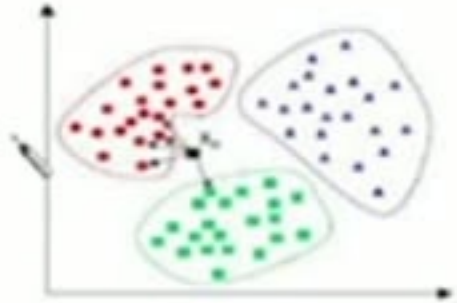


Try to update model parameters (Iteratively) to minimize error (Loss Function)

Nearest Neighbor (NN)
and
K -Nearest Neighbor (K -NN)

K -Nearest Neighbor (K -NN)

- **Given:** training data D = $\{(x_1, y_1), \ldots, (x_N, y_N)\}$ and a test point
- **Prediction Rule:** Look at the K most similar training examples



  - For **classification:** assign the majority class label (**majority voting**)
  - For regression: assign the average response
- The algorithm requires:
  - **Parameter K** : number of nearest neighbors to look for

K -Nearest Neighbors Algorithm

- **Compute** the test point's distance from each training point
- **Sort** the distances in ascending (or descending) order
- Use the sorted distances to **select the K nearest neighbors**
- Use **majority rule** (for classification) or averaging (for regression)

Note: K -Nearest Neighbors is called a non-parametric method

- Unlike other supervised learning algorithms,

  K -Nearest Neighbors doesn't:

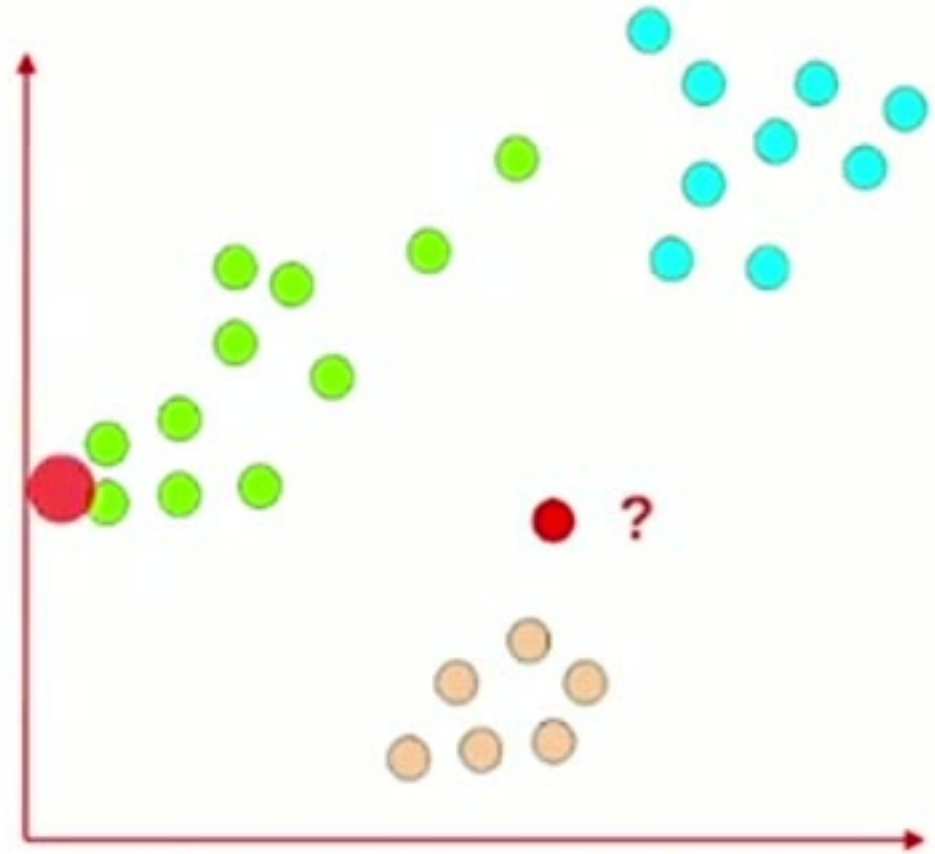
  - learn an explicit mapping f from the training data
  - It simply uses the training data at the test time to make predictions

K-NN: Computing the distances

- The K-NN algorithm requires computing distances of the test example from each of the training examples
- Several ways to compute distances
- The choice depends on the type of the features in the data
- Real-valued features ($x_i \in R^D$): **Euclidean** distance is commonly used

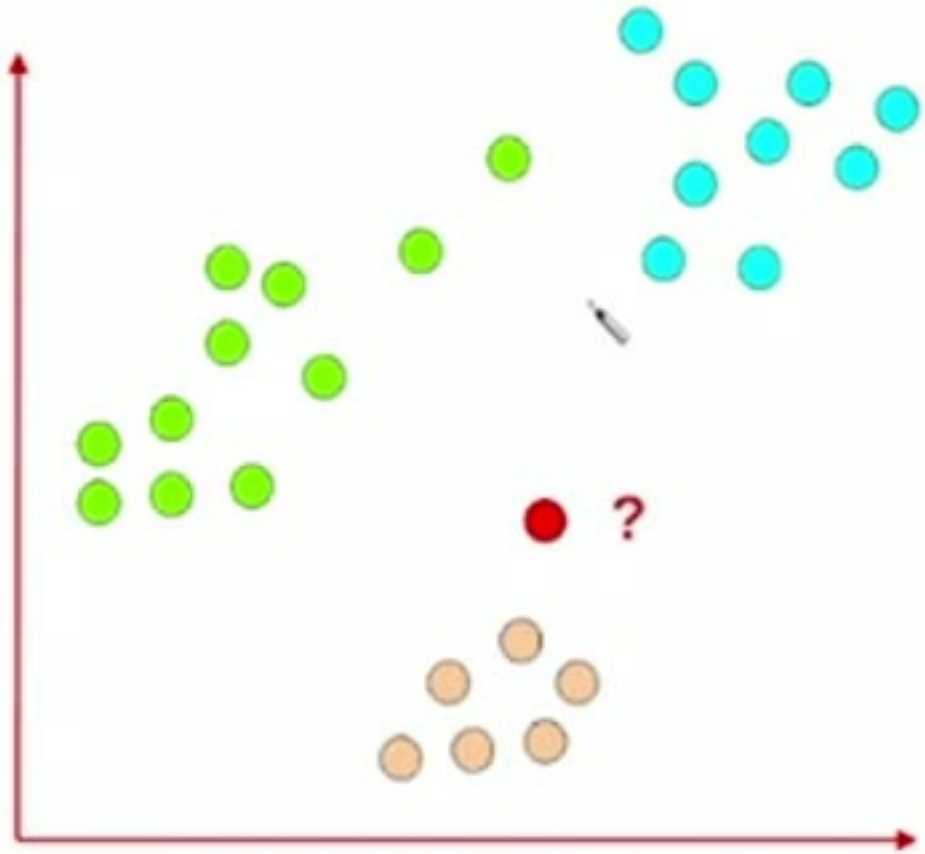
$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^{D} (x_{im} - x_{jm})^2} = \sqrt{||\mathbf{x}_i||^2 + ||\mathbf{x}_j||^2 - 2\mathbf{x}_i^T \mathbf{x}_j}$$

Nearest Neighbors (NN) Example

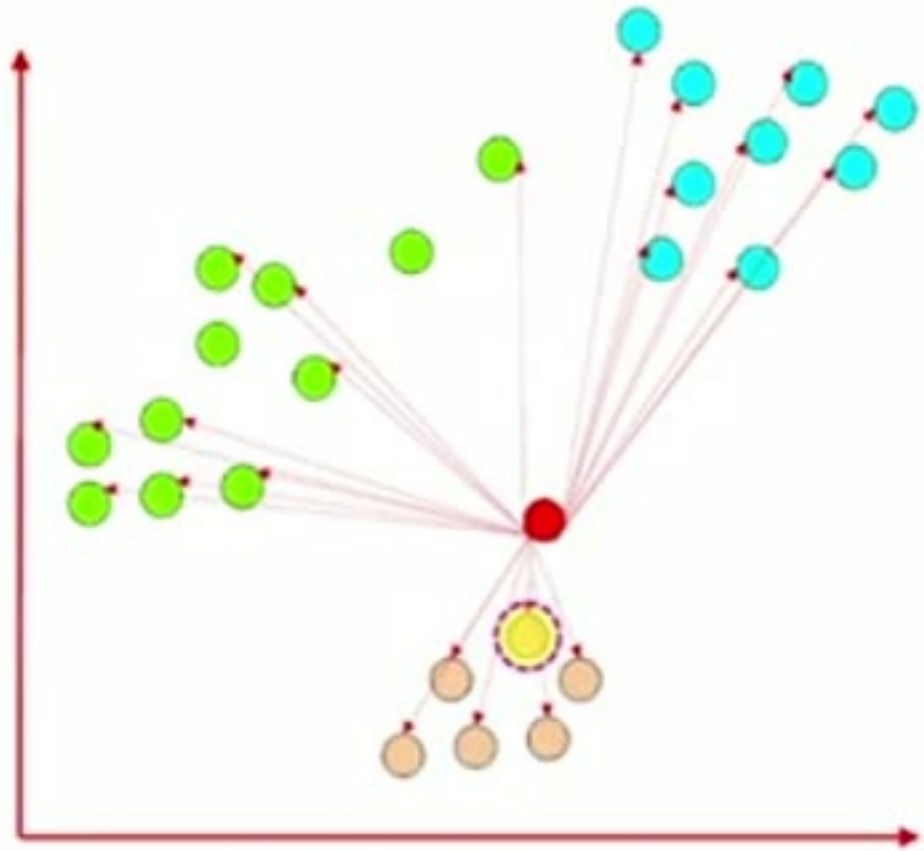


Data of Three Classes and ONE TEST sample

Nearest Neighbors (NN) Example

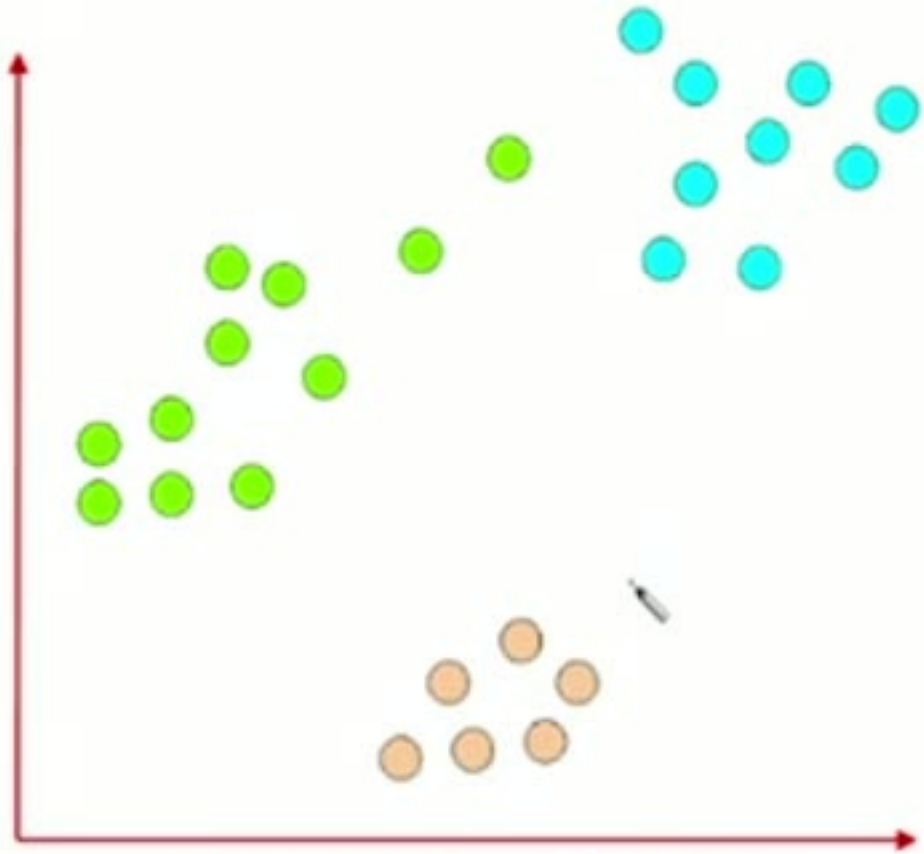


Data of Three Classes

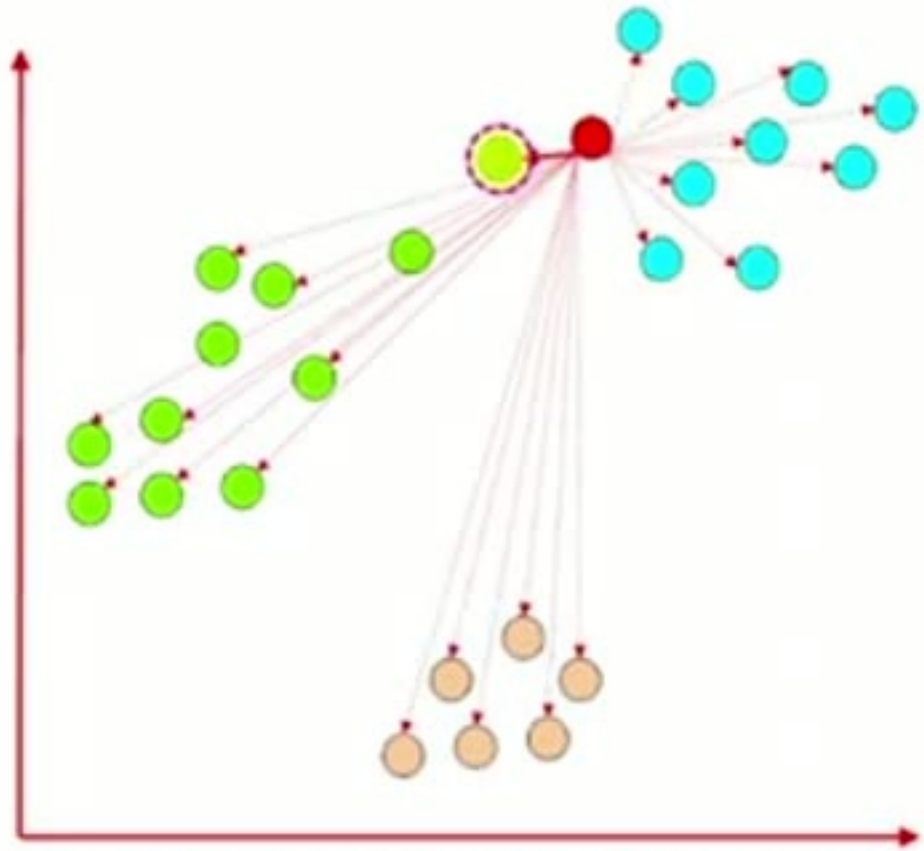


Nearest Neighbor is the Orange class

Nearest Neighbors (NN) Outlier Problem

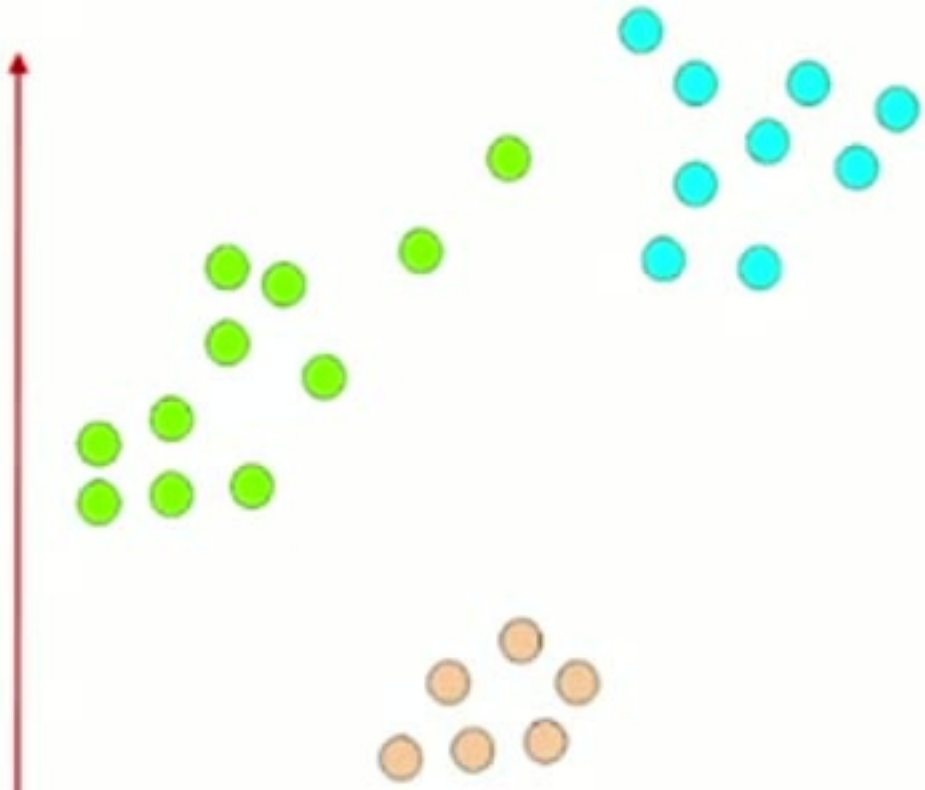


Data of Three Classes

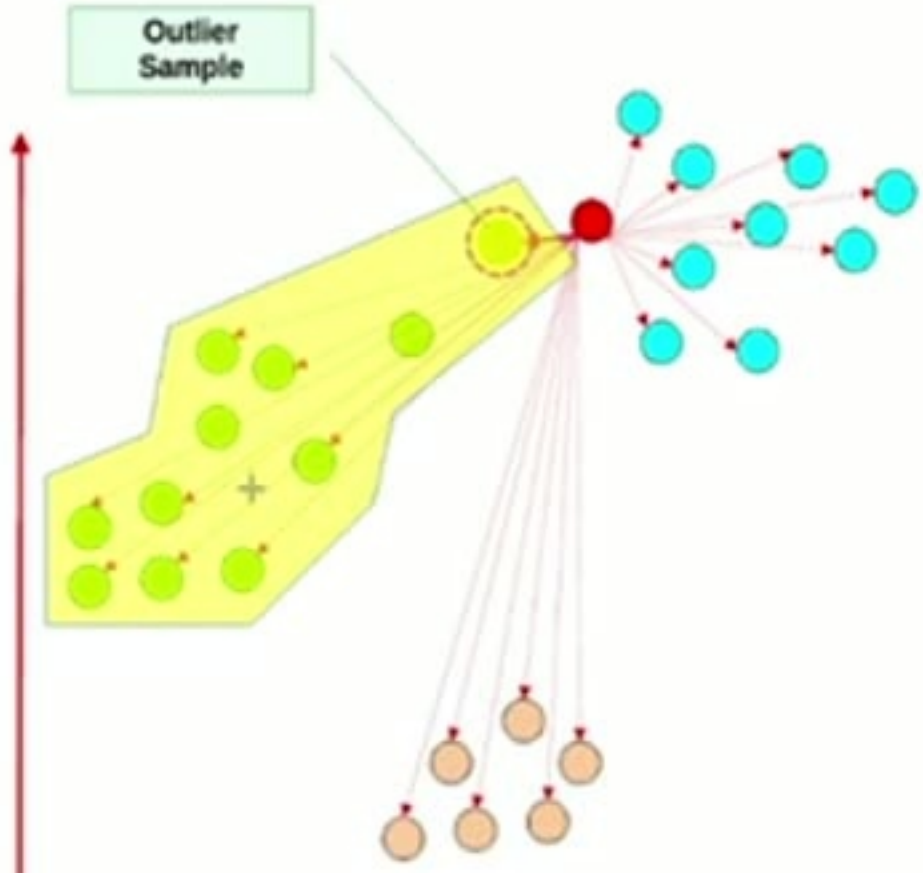


Nearest Neighbor is the Green class

Nearest Neighbors (NN) Outlier Problem

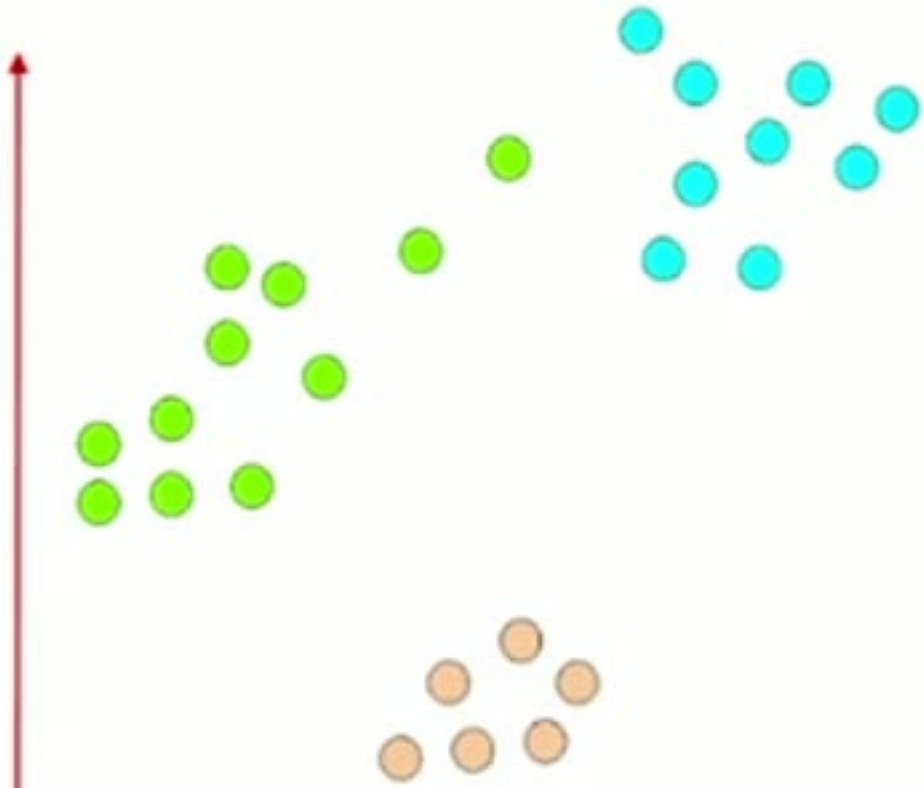


Data of Three Classes

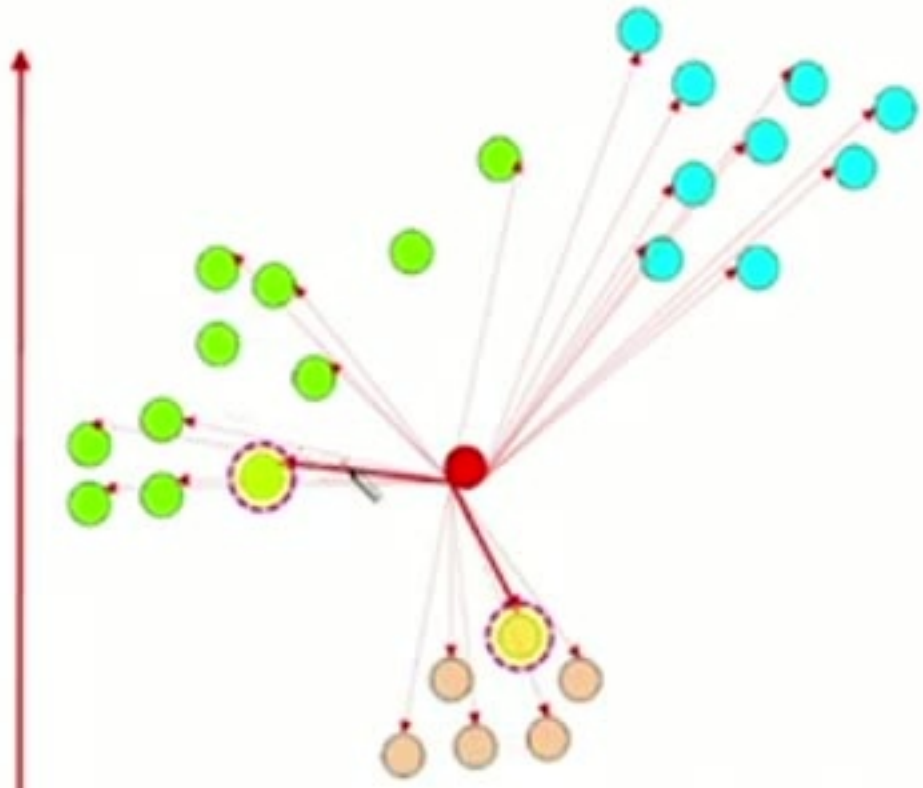


Nearest Neighbor is the Green class

Nearest Neighbors (NN) Equal Distance

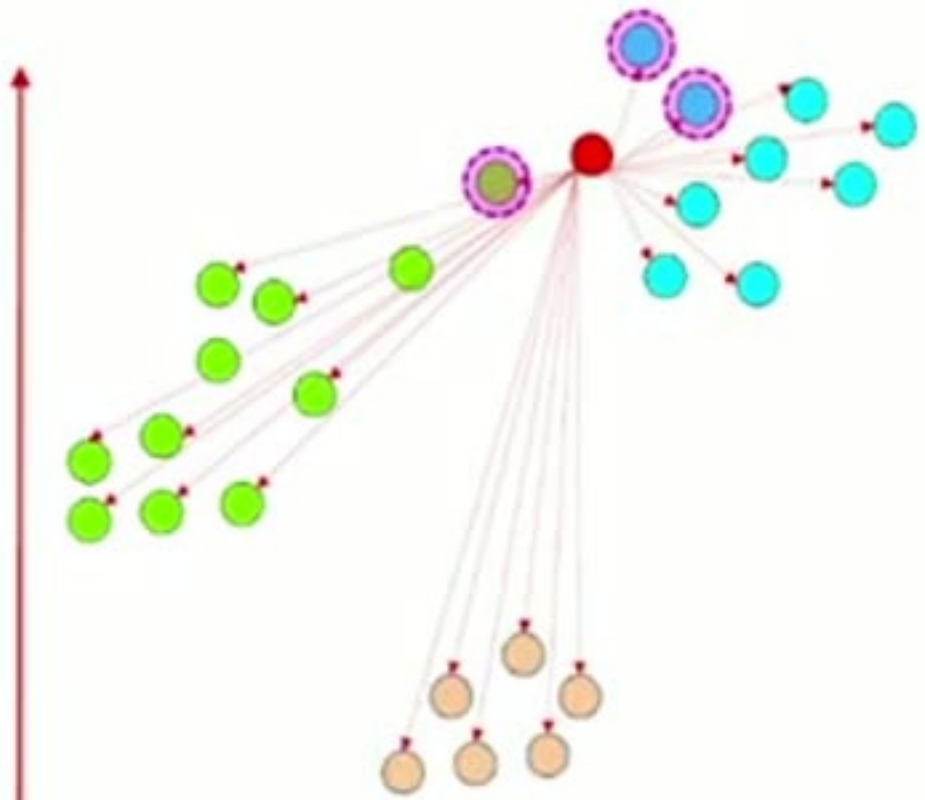


Data of Three Classes

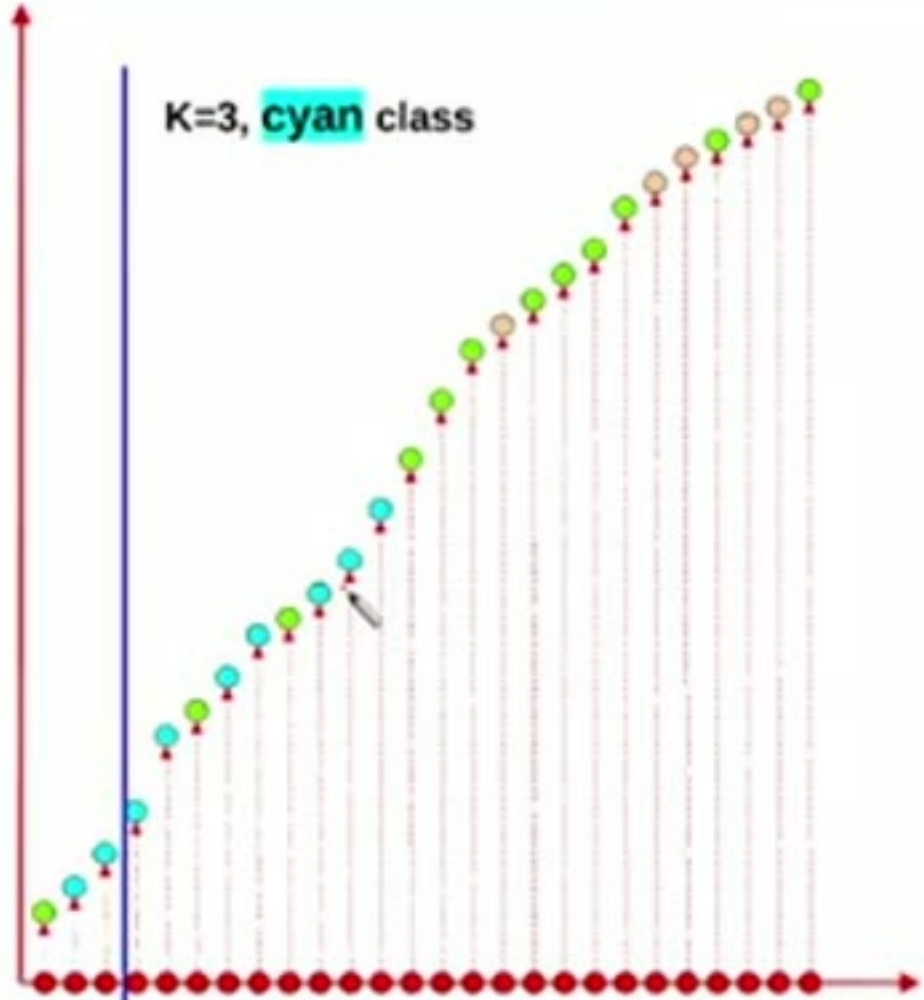


No Unique Nearest Neighbor

K-Nearest Neighbors (KNN) , K=3



Select Nearest 3 Neighbors

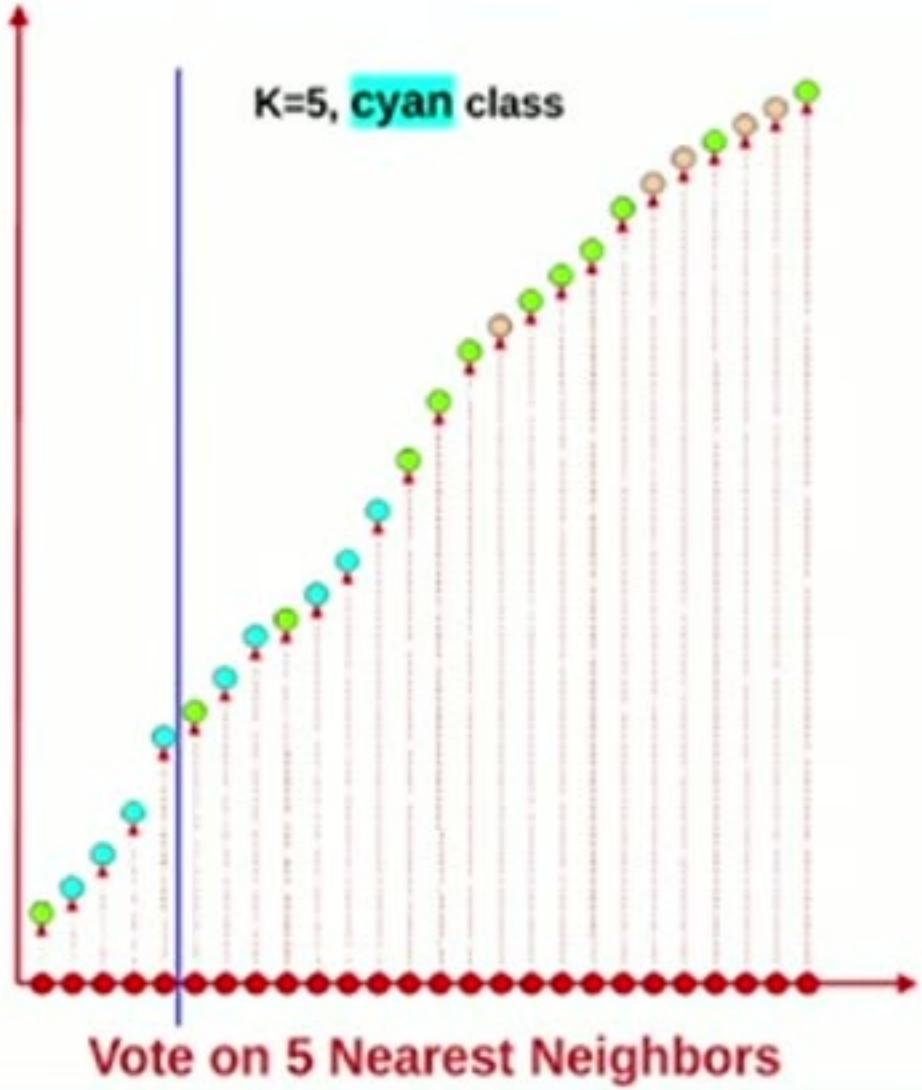


Vote on 3 Nearest Neighbors

K-Nearest Neighbors (KNN) , K=5



Select Nearest 5 Neighbors

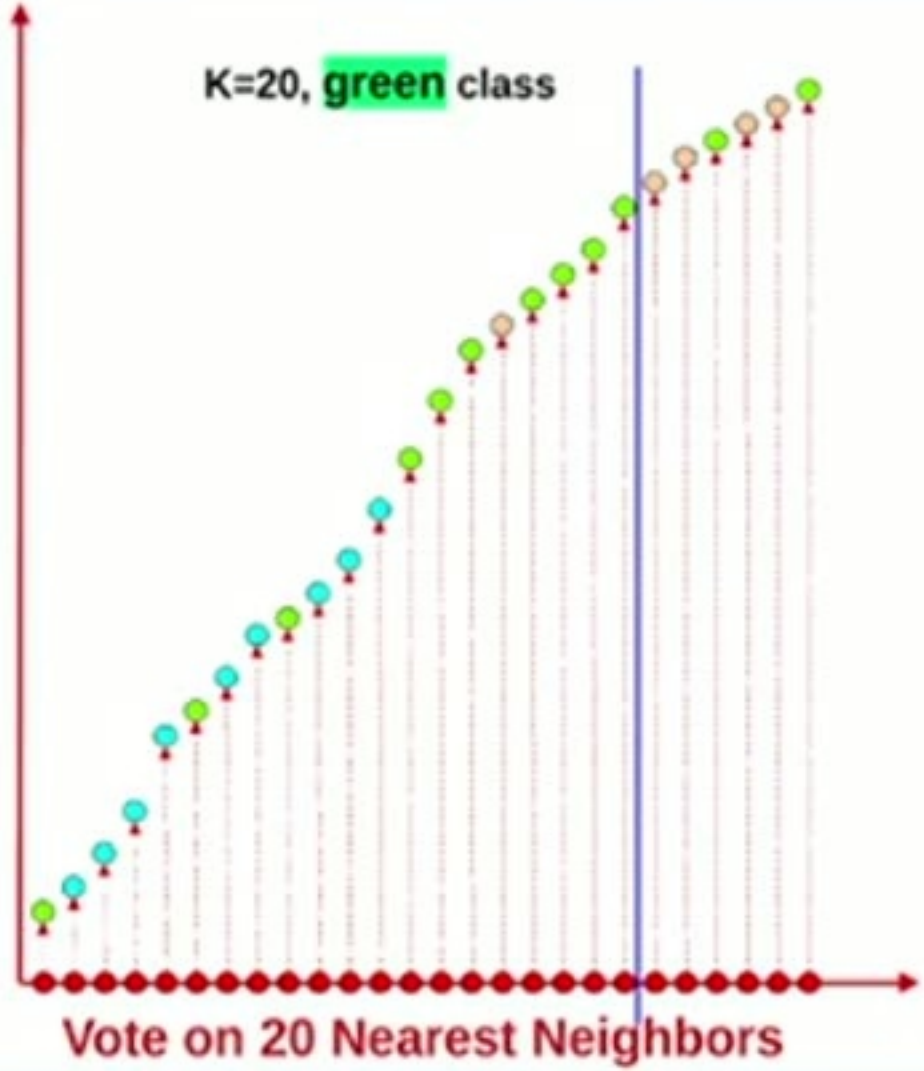


Vote on 5 Nearest Neighbors

K-Nearest Neighbors (KNN) , K=20

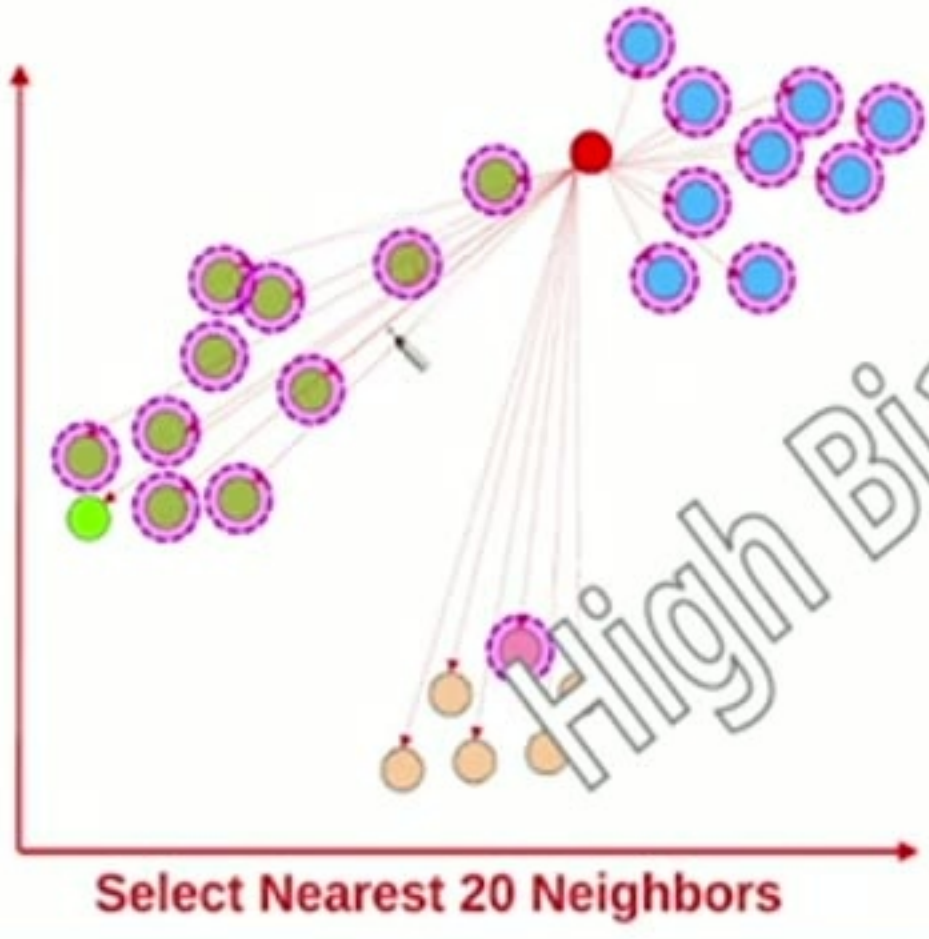


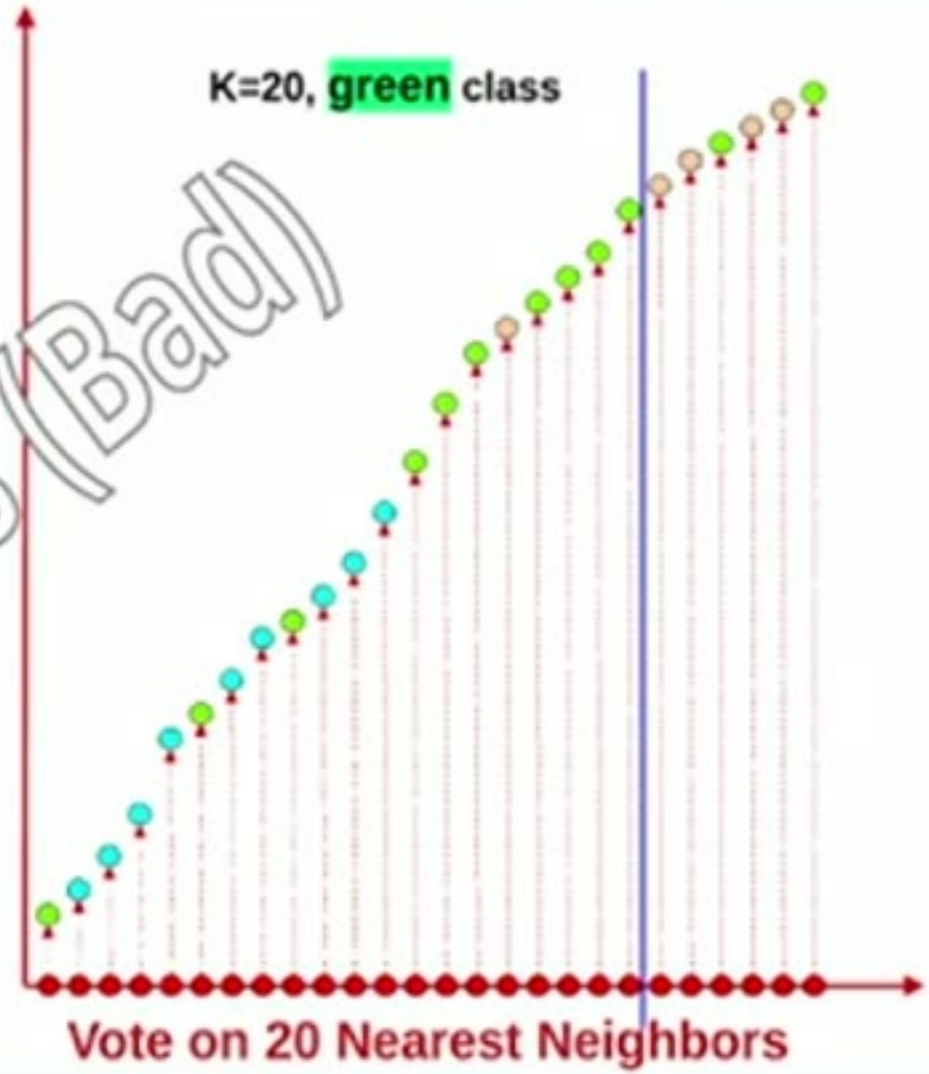
Select Nearest 20 Neighbors



Vote on 20 Nearest Neighbors

K-Nearest Neighbors (KNN) , K=20





High Bias (Bad)

K-NN: Feature Normalization

Features should be on the **same scale**

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^{D} (x_{im} - x_{jm})^2}$$

- Example: if one feature has its values in millimeters and another has in centimeters, we would need to normalize
- One way is:

Replace x_{im} by $z_{im} = \frac{(x_{im} - \bar{x_m})}{\sigma_m}$ (make them zero mean, unit variance)

$\bar{x_m} = \frac{1}{N} \sum_{i=1}^{N} x_{im}$: empirical mean of mth feature

$\sigma_m^2 = \frac{1}{N} \sum_{i=1}^{N} (x_{im} - \bar{x_m})^2$: empirical variance of mth feature

Example: Distance without Normalization

Feature Vector of bars:

	Red Class			Blue Class			Green Class			Test
Weight (KG)	2	3	4	7	8	9	12	13	14	7
Length (cm)	100	150	200	150	200	250	150	250	300	120

Squared distance between each vector and test vector

Red Class			Blue Class			Green Class		
425	916	6409	900	6401	16904	925	16936	32449

$\begin{pmatrix} 7 \\ 100 \end{pmatrix}$ Belongs to Red Class

Example: Distance with Normalization

Feature Vector for bars:

	Red Class			Blue Class			Green Class			Mean	S.D
Weight (KG)	2	3	4	7	8	9	12	13	14	8	4.163
Length (cm)	100	150	200	150	200	250	150	250	300	194.44	59.835

	Normalized Red Class			Normalized Blue Class			Normalized Green Class			Normalized Test
Weight (KG)	−1.441	−1.201	−0.961	−0.24	0	0.24	0.961	1.201	1.441	−0.24
Length (cm)	−1.578	−0.743	0.093	−0.743	0.093	0.928	−0.743	0.928	1.764	−1.244

Squared distance between each Normalized vector and Normalized test vector

1.554	1.175	2.307	0.251	1.845	4.948	1.693	6.794	11.874

$$Normalized\ X = \frac{X - Mean}{S.D.}$$

$\begin{pmatrix} 7 \\ 100 \end{pmatrix}$ Belongs to Blue Class

K -NN: Some other distance measures

- **Binary-valued features**
  - Use Hamming distance:

$$d(x_i, x_j) = \sum_{m=1}^{D} \mathbb{I}(x_{im} \neq x_{jm})$$

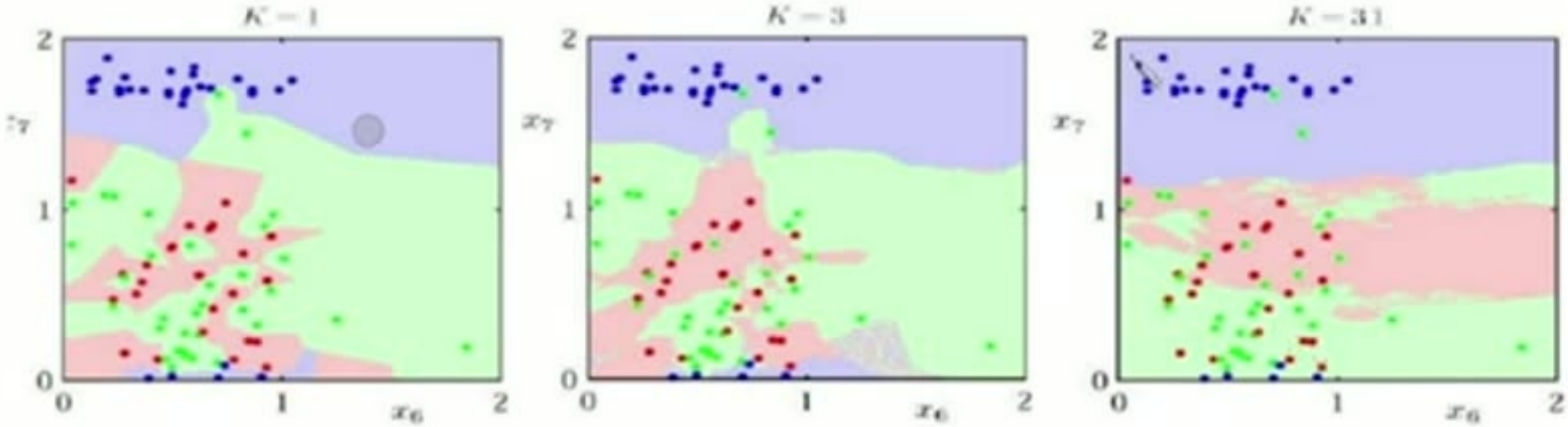
  - Hamming distance counts the number of features where the two examples disagree
- **Mixed feature types** (some real-valued and some binary-valued)?
  - Can use mixed distance measures
  - E.g., **Euclidean** for the real part, **Hamming** for the binary part
- Can also *assign weights to features*:

$$d(x_i, x_j) = \sum_{m=1}^{D} w_m d(x_{im}, x_{jm})$$

Choice of K - Neighborhood Size

- **Small K**
  - Creates many small regions for each class
  - May lead to non-smooth decision boundaries **and over-fit**
- **Large K**
  - Creates fewer larger regions
  - Usually leads to smoother decision boundaries (caution: too smooth decision boundary can **under-fit**). **High bias** to classes with larger members
- **Choosing K**
  - Often data dependent and heuristic based
  - Or using cross-validation (using some held-out data)
  - **In general, a K too small or too big is bad!**
-

Choice of K - Neighborhood Size



K -Nearest Neighbor: Properties

What's nice

- Simple and intuitive; easily implementable
- K -NN approaches the best possible classifier (Bayes optimal)

What's not so nice..

- **Store all the training data in memory even at test time**
  - Can be memory intensive for large training datasets
  - An example of non-parametric, or memory/instance-based methods
  - Different from parametric, model-based learning models
- **Expensive at test time: O(ND) computations for each test point**
  - Have to search through all training data to find nearest neighbors
  - Distance computations with N training points (D features each)
- **Sensitive to noisy features**
- **May perform badly in high dimensions (curse of dimensionality)**

KNN Assignment

1) Download "IRIS" Data set (three flowers) [150 Samples]
2) Use PCA to visualize the Dataset (as 2-D) *[Optional]*
3) Divide the data into 80% Training, 20% test [120 , 30 samples]
4) Apply KNN with K=3, 5, and 7 on Test Samples.
5) Calculate the classification accuracy for each K, Which is the best?
6) **Normalize** the IRIS dataset and repeat steps 4 and 5. Compare with previous results, does normalization affect the accuracy ?

IRIS Dataset

iris setosa



iris versicolor

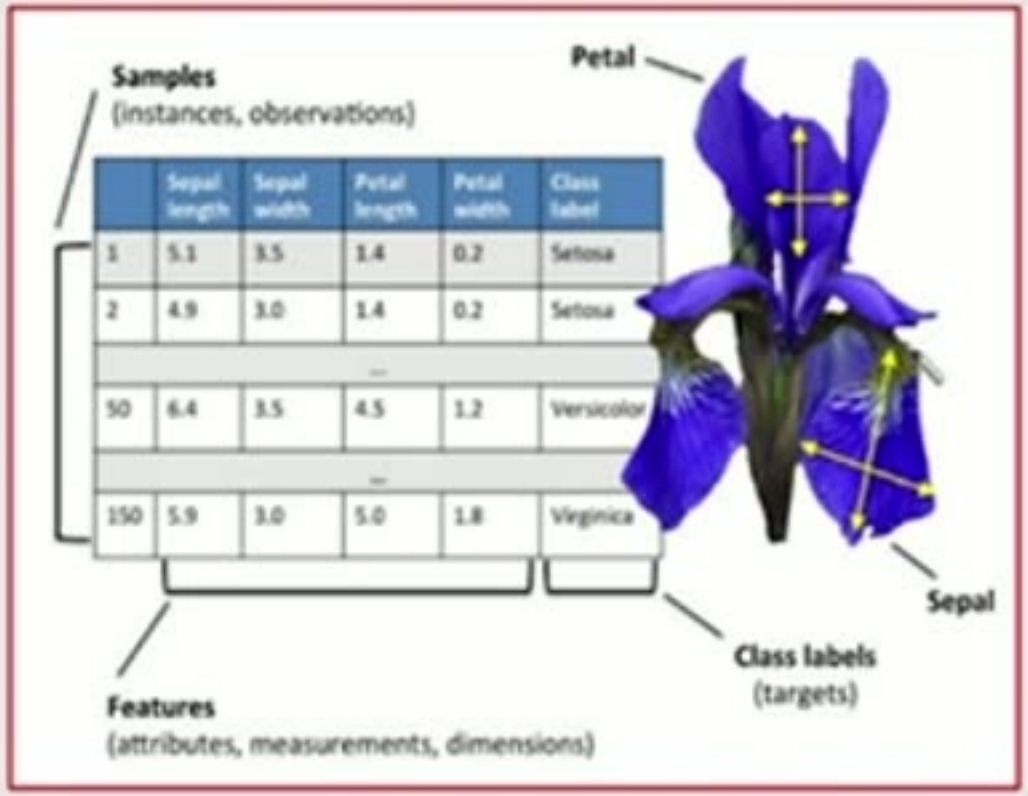


iris virginica



150 Samples, 3 Classes, features of petal and sepal parts of each flower

Example of High Dimensional Data (IRIS Dataset)



	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica



4 Features: Width and Length of both sepal and petal

IRIS Dataset

Serial	sepal.length	sepal.width	petal.length	petal.width	Class
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3	1.4	0.2	Setosa
3	4.7	3.2	1.3	0.2	Setosa
4	4.6	3.1	1.5	0.2	Setosa
5	5	3.6	1.4	0.2	Setosa
6	5.4	3.9	1.7	0.4	Setosa
7	4.6	3.4	1.4	0.3	Setosa
8	5	3.4	1.5	0.2	Setosa
					. .
					. .
45	5.1	3.8	1.9	0.4	Setosa
46	4.8	3	1.4	0.3	Setosa
47	5.1	3.8	1.6	0.2	Setosa
48	4.6	3.2	1.4	0.2	Setosa
49	5.3	3.7	1.5	0.2	Setosa
50	5	3.3	1.4	0.2	Setosa
51	7	3.2	4.7	1.4	Versicolor
52	6.4	3.2	4.5	1.5	Versicolor
53	6.9	3.1	4.9	1.5	Versicolor
54	5.5	2.3	4	1.3	Versicolor
55	6.5	2.8	4.6	1.5	Versicolor
56	5.7	2.8	4.5	1.3	Versicolor
					. .
					. .
96	5.7	3	4.2	1.2	Versicolor
97	5.7	2.9	4.2	1.3	Versicolor
98	6.2	2.9	4.3	1.3	Versicolor
99	5.1	2.5	3	1.1	Versicolor
100	5.7	2.8	4.1	1.3	Versicolor
101	6.3	3.3	6	2.5	Virginica
102	5.8	2.7	5.1	1.9	Virginica
103	7.1	3	5.9	2.1	Virginica
104	6.3	2.9	5.6	1.8	Virginica
					. .
					. .
148	6.5	3	5.2	2	Virginica
149	6.2	3.4	5.4	2.3	Virginica
150	5.9	3	5.1	1.8	Virginica

150 Samples

3 Classes

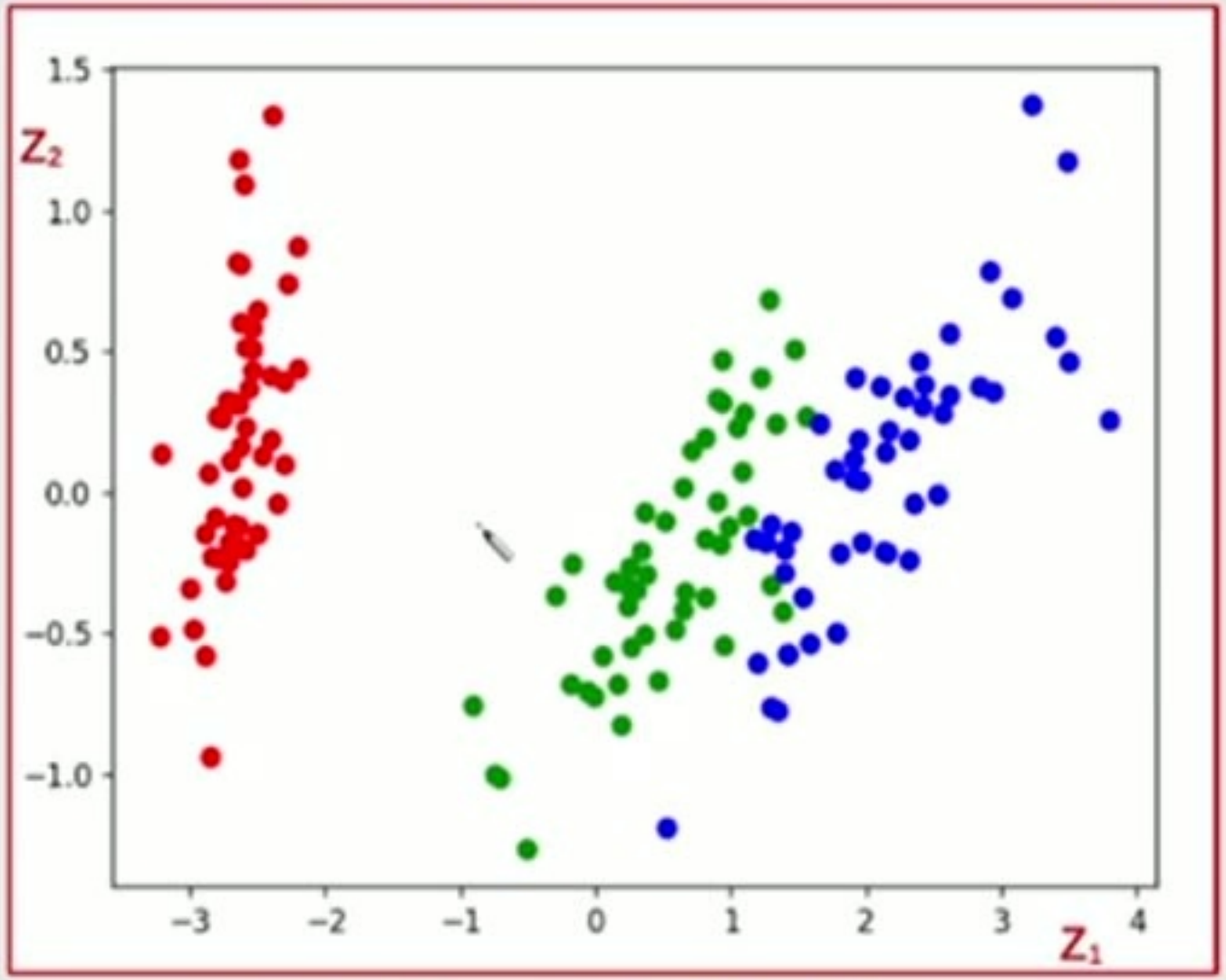
Setosa
Versicolor
Virginica

4 Features

No Direct Visualization

Visualize Reduced Features [4 features are reduced to two features]

$$\begin{pmatrix} W_1 \\ L_1 \\ W_2 \\ L_2 \end{pmatrix} \xrightarrow{PCA} \begin{pmatrix} Z_1 \\ Z_2 \end{pmatrix}$$

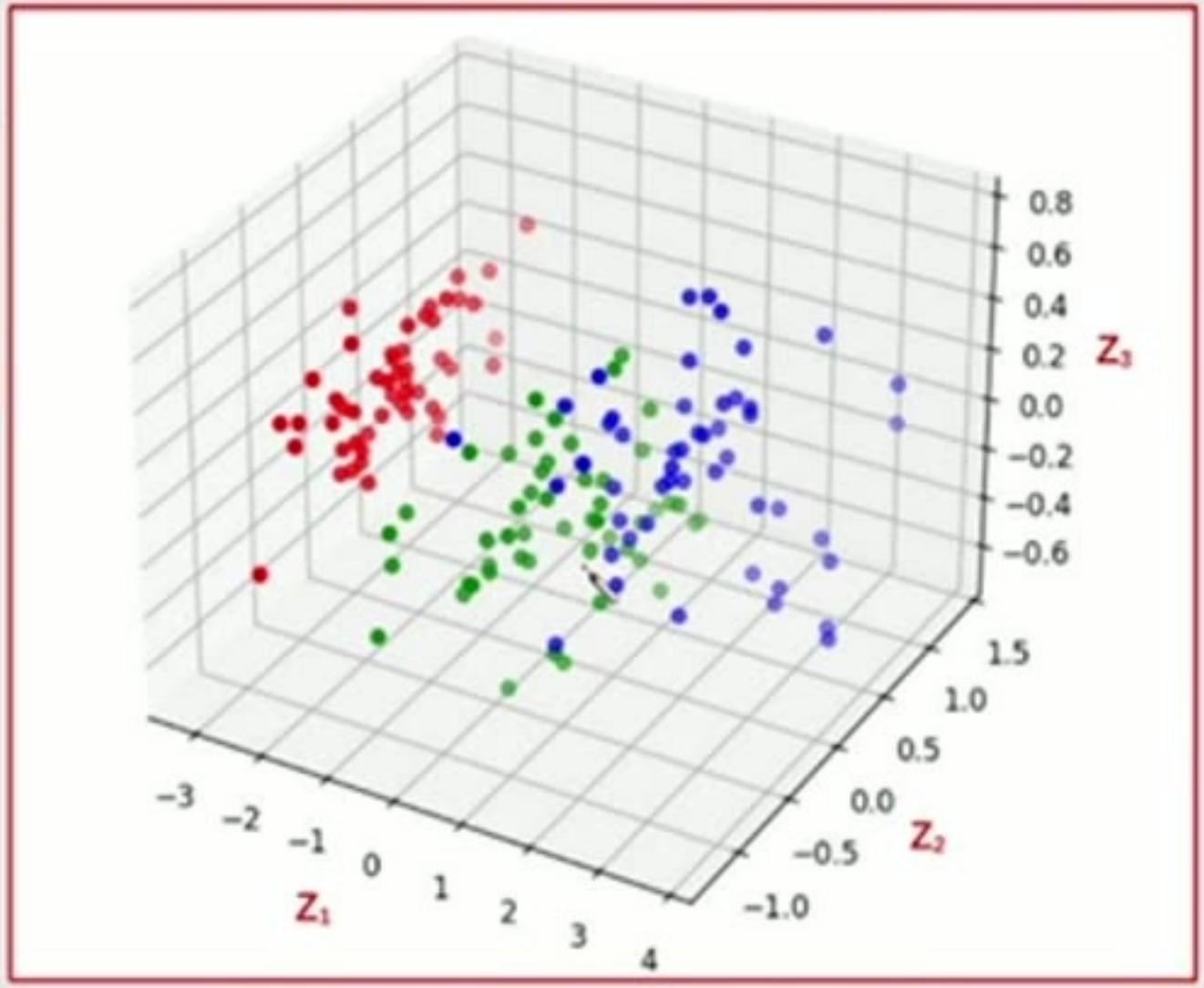


Setosa
Versicolor
Virginica

Visualize Classes as 2-D (Z_1 vs Z_2)

[3] Visualize Reduced Features [4 features are reduced to three features]

$$\begin{pmatrix} W_1 \\ L_1 \\ W_2 \\ L_2 \end{pmatrix} \xrightarrow{PCA} \begin{pmatrix} Z_1 \\ Z_2 \\ Z_3 \end{pmatrix}$$



Setosa
Versicolor
Virginica

Visualize Classes as 3-D (z_1 vs Z_2 vs Z_3)